ISLAMIC CALL UNIVERSITY COLLEGE
Faculty of Science — Department of Computer Science

CSC 3105

Computer Organization and Architecture

Module 2

INSTRUCTION SET: Data Transfer, Arithmetic Operations, Interrupt Control, and Subroutine Calls

Year of Study: III | Credit Hours: 4 | Allocated Time: 8 Hours

Lecture Notes

Learning Objectives

By the end of this module, students should be able to:

1. Explain what an instruction set architecture (ISA) is and describe the general structure of a machine instruction.
2. Identify and simulate the execution of data transfer (copy) instructions between registers and memory.
3. Identify and simulate the execution of arithmetic instructions — addition, subtraction, multiplication and division — and explain their effect on the processor's status flags.
4. Explain the purpose of the Interrupt Enable flag and describe how instructions that enable or disable interrupts alter processor behaviour.
5. Explain the mechanics of an unconditional CALL instruction, including its use of the stack, and contrast it with conditional CALL instructions.

2.1 Introduction to the Instruction Set

The instruction set of a processor is the complete vocabulary of operations that the processor's Control Unit is wired (or, in some designs, microprogrammed) to recognise and carry out. Just as a natural language has a fixed set of words with agreed meanings, a processor's instruction set is a fixed catalogue of bit patterns, each of which has one, and only one, agreed meaning to that particular family of processors. This catalogue is commonly referred to by the abbreviation ISA — Instruction Set Architecture — because it defines the architecture as seen by anyone writing software for the machine, independently of how the underlying electronic circuitry happens to be built.

Every machine instruction, regardless of the operation it performs, is generally composed of two conceptual parts:

- **Opcode (operation code):** a unique bit pattern identifying which operation — add, move, call, and so on — is to be performed.
- **Operand(s):** the data, register names, or memory addresses that the operation is to act upon. An instruction may have zero, one, two, or occasionally three operands, depending on the operation and the processor design.

Instruction sets are conventionally grouped into functional categories, and this module addresses three of the most important such categories in turn: data transfer instructions,

which copy information from one location to another without altering it; arithmetic instructions, which compute new values from existing ones; and program-control instructions — specifically CALL and its conditional variants — which alter the normal, sequential flow of execution established in Module 1.

> **Everyday analogy**
> If the microprocessor is the clerk described in Module 1, the instruction set is the fixed list of actions the clerk knows how to perform on request — 'copy this document', 'add these two numbers', 'phone this other office and wait for them to finish before continuing'. The clerk cannot invent a new action on the spot; every instruction executed must be one from this predefined list, expressed in the exact bit-pattern 'vocabulary' the clerk was trained to recognise.

2.1.1 Note on Notation Used in This Module

The examples in this module use a generic 8-bit assembly-language notation, broadly representative of classic microprocessor families, to keep the underlying concepts independent of any single vendor's syntax. A, B, C, D, H and L denote general-purpose or accumulator registers; M denotes the memory location addressed by the H–L register pair; and flags referred to are the Zero (Z), Carry (CY), Sign (S) and Parity (P) flags of the processor's status (flag) register. The same principles apply, with only syntactic differences, to any other instruction set your simulator or textbook may use.

2.2 Data Transfer (Copy) Instructions

A data transfer instruction copies the contents of a source location into a destination location without modifying the source and, in most instruction sets, without affecting the processor's status flags. Despite performing no computation, data transfer instructions are typically the most frequently executed instructions in any real program, since almost every arithmetic or logical operation must first bring its operands into a register and later move the result to where it is needed.

2.2.1 Register-to-Register Transfer

The simplest data transfer copies the contents of one register directly into another, entirely within the processor and without any bus activity beyond what the opcode fetch itself required.

```
; Copy the contents of register B into register A
MOV A, B     ; A now holds a copy of B; B is unchanged
```

Simulated step by step: before execution, suppose register A holds 00H and register B holds 3CH. During the single machine cycle this instruction requires (an opcode fetch — no memory operand is involved), the Control Unit routes the output of register B directly into the input of register A over the processor's internal data path. After execution, A holds 3CH and B still holds 3CH; the transfer is a copy, not a move, in the sense that the source retains its original value.

2.2.2 Register-to-Memory and Memory-to-Register Transfer

When a source or destination is a memory location rather than a register, the transfer additionally requires a Memory Read or Memory Write machine cycle of the kind studied in Module 1, since the value must physically cross the address and data buses.

```
; Load the accumulator from the memory address held in H-L
MOV A, M     ; Memory Read cycle: A ← contents of address (H)(L)

; Store the accumulator into the memory address held in H-L
MOV M, A     ; Memory Write cycle: contents of address (H)(L) ← A
```

2.2.3 Immediate Data Transfer

An immediate transfer loads a register directly with a constant value that is embedded in the instruction stream itself, immediately following the opcode. As explained in Module 1, this requires an additional operand-fetch machine cycle beyond the initial opcode fetch.

```
MVI C, 1FH    ; C ← 1FH (two machine cycles: opcode fetch + operand fetch)
```

Practical example

A payroll program needs to bring an employee's basic pay, held in memory, into the accumulator so that a bonus can later be added to it. The instruction MOV A, M — with the H-L register pair already loaded with the pay record's address — performs exactly this transfer. No arithmetic has yet taken place and no flag is affected; the accumulator is simply now holding the same value that memory holds, ready for the next, arithmetic, instruction to act on it.

2.3 Arithmetic Instructions

Arithmetic instructions direct the Arithmetic and Logic Unit to compute a new value from one or two existing values, and — unlike data transfer instructions — they characteristically update one or more bits of the processor's status (flag) register to reflect properties of the result, such as whether it is zero, negative, or too large to fit in the

destination register. This section examines addition, subtraction, multiplication and division in turn, in each case simulating a concrete example instruction by instruction.

2.3.1 Addition (ADD)

An ADD instruction directs the ALU to add the contents of a specified register (or memory location) to the accumulator, leaving the sum in the accumulator. Addition affects the Zero flag (set when the result is 00H), the Carry flag (set when the addition produces a result too large to fit in eight bits), the Sign flag, and the Parity flag.

```
; Add the contents of register B to the accumulator
ADD B          ; A ← A + B
```

Step	Register A	Register B	Flags Affected	Result
Before execution	3AH	05H	—	—
ALU computes 3AH + 05H	—	—	—	3FH
After execution	3FH	05H	Z=0, CY=0, S=0	A holds 3FH

Figure 2.1 — Simulated execution of ADD B, showing register contents before and after the operation and the flags set as a side-effect.

> **Practical example**
> A simple point-of-sale routine keeps a running total of a customer's purchases in the accumulator. Each time an item is scanned, its price (already placed in register B by an earlier data-transfer instruction) is added to the running total with ADD B. If a particular addition causes the eight-bit total to exceed 255, the Carry flag is automatically set, which the program can test afterwards to decide whether to extend the total into a second byte — this is exactly the mechanism used to build arithmetic on numbers wider than the processor's native register size.

2.3.2 Subtraction (SUB)

A SUB instruction subtracts the contents of a specified register (or memory location) from the accumulator, again leaving the result in the accumulator. Internally, most processors implement subtraction by adding the two's-complement (negated) form of the subtrahend to the accumulator, so the same adder circuitry inside the ALU used for ADD is reused for SUB. The Carry flag, in the context of subtraction, is conventionally interpreted as a 'borrow' flag: it is set when the value being subtracted is larger than the value in the accumulator.

```
; Subtract the contents of register C from the accumulator
SUB C          ; A ← A - C
```

Step	Register A	Register C	Flags Affected	Result
Before execution	10H	18H	—	—
ALU computes 10H − 18H	—	—	—	F8H (borrow occurred)
After execution	F8H	18H	Z=0, CY=1 (borrow), S=1	A holds F8H

Figure 2.2 — Simulated execution of SUB C. Because 18H is larger than 10H, the subtraction borrows, and the Carry flag is set to indicate this.

2.3.3 Multiplication (MUL)

Multiplication is computationally more demanding than addition or subtraction, and historically many microprocessor families did not provide a single native multiply instruction at all — the operation was instead built up in software from repeated addition and shifting. Processors that do provide a native MUL instruction typically multiply the accumulator by a specified register and place the result, which may require twice as many bits as either operand, across a pair of registers rather than a single one.

```
; Multiply the accumulator by the contents of register D
MUL D          ; (H,L) or (A,extension) ← A × D — a double-width result
```

Step	Register A	Register D	Product	Notes
Before execution	0AH (10 decimal)	0CH (12 decimal)	—	—
ALU computes 10 × 12	—	—	120 decimal = 0078H	Fits comfortably in 16 bits
After execution	—	—	High byte 00H, low byte 78H	Result stored across a register pair

Figure 2.3 — Simulated execution of MUL D. The product of two 8-bit values may require up to 16 bits, so it is stored across two registers.

Software alternative on processors without a MUL instruction

On an instruction set that has no native multiply, the same 10 × 12 calculation can be built from a loop that adds the multiplicand to a running total once for every unit in the multiplier — that is, adding 10 to an initially-zero total, twelve times in succession, decrementing a loop counter after each addition and repeating until the counter reaches zero. This 'repeated addition' technique is a standard laboratory exercise for appreciating why a dedicated MUL instruction, when available, is considerably faster than software emulation.

2.3.4 Division (DIV)

Division is the most demanding of the four basic arithmetic operations and, like multiplication, is not provided as a native instruction on every processor family. Where a DIV instruction is available, it typically divides a double-width dividend (spread across a register pair) by the contents of a specified register, leaving a single-width quotient in one register and the remainder in another.

```
; Divide the (H,L) register pair by the contents of register E
DIV E        ; A ← quotient, B ← remainder
```

Step	Dividend (H,L)	Register E	Quotient	Remainder
Before execution	0043H (67 decimal)	08H (8 decimal)	—	—
ALU computes 67 ÷ 8	—	—	8	3
After execution	—	08H	A = 08H	B = 03H

Figure 2.4 — Simulated execution of DIV E, producing both a quotient and a remainder in separate registers.

Practical example

A simple embedded controller needs to convert a raw sensor reading of 67 units into 'whole packs of 8' for display, together with any leftover units. Loading the H-L pair with 0043H (67 decimal) and register E with 08H, then executing DIV E, leaves the whole number of packs (8) in the accumulator and the leftover units (3) in register B — precisely the two pieces of information the display routine needs, obtained in a single instruction rather than a manual repeated-subtraction loop.

It is worth noting, as a point of caution for both simulation exercises and real programming, that a division instruction where the divisor is zero is undefined or

produces an error condition on virtually every processor family, and well-written programs test a divisor for zero before executing a DIV instruction.

2.4 Enabling and Disabling Interrupts

Module 1 introduced the interrupt as a mechanism by which an external device can divert the processor's attention from the program it is currently running. That mechanism, however, is only effective if the processor is, at the relevant moment, willing to listen for such requests. Every processor that supports maskable interrupts maintains a single internal flip-flop — the Interrupt Enable (IE) flag — which the Control Unit consults at the fixed checkpoint, described in Module 1, at the end of every instruction cycle. If the Interrupt Enable flag is clear, any pending maskable interrupt request is simply ignored until the flag is later set; if it is set, a pending request is honoured and the interrupt-acknowledge sequence begins.

2.4.1 The EI and DI Instructions

Two short, operand-less instructions are provided specifically to manipulate this flag under direct program control:

- **EI (Enable Interrupts):** sets the Interrupt Enable flip-flop, permitting maskable interrupt requests to be honoured from this point onward.
- **DI (Disable Interrupts):** clears the Interrupt Enable flip-flop, causing maskable interrupt requests to be ignored until a subsequent EI is executed.

```
DI              ; disable maskable interrupts
; --- critical section: update a shared two-byte counter ---
LXI H, 2000H
INX H
; --- end critical section ---
EI              ; re-enable maskable interrupts
```

It is important to note one architectural safeguard common to many processor families: because an interrupt occurring in the middle of the two-instruction sequence used to re-enable interrupts and then immediately act on new data could itself be disruptive, the effect of EI is often deliberately delayed until after the instruction immediately following it has completed, giving the programmer a brief, guaranteed interrupt-free instruction in which to, for example, return from a service routine cleanly. Consult the specific data sheet of the processor family in use for the exact timing guarantee it provides.

2.4.2 Why Programs Deliberately Disable Interrupts

At first glance it may appear that interrupts should always be left enabled, since their entire purpose is to let the processor respond promptly to external events. There are, however, short sequences of instructions — critical sections — during which an interruption would leave data in an inconsistent state if it were allowed to occur partway through. A common example is updating a multi-byte value that is also read by an interrupt service routine: if the low byte has been updated but the high byte has not yet been, an interrupt occurring at that exact instant and reading both bytes would see a value that never actually existed. Surrounding such a sequence with DI and EI, as shown above, guarantees that it runs to completion without interference.

> **Practical example**
> A real-time clock service routine increments a two-byte 'seconds elapsed' counter held in memory every time a hardware timer interrupt occurs. A separate part of the main program periodically reads this same counter to display the elapsed time. If the main program read the counter's two bytes exactly when the timer interrupt fired and updated them, it could read a half-updated value — for example, reading a newly-rolled-over low byte of 00H together with an old, not-yet-incremented high byte. Wrapping the main program's read of the counter in DI … EI guarantees the timer interrupt cannot fire midway through the read, at the cost of the timer being very briefly, and harmlessly, delayed if it happens to occur during that short window.

2.5 CALL and Conditional CALL Instructions

A CALL instruction transfers execution to a different point in the program — the beginning of a subroutine — while automatically preserving the information needed to resume the original sequence once the subroutine has finished. This is the fundamental mechanism by which programs are organised into reusable procedures rather than a single, undifferentiated block of sequential code, and it is the software counterpart of the hardware interrupt-acknowledge sequence studied in Module 1: both mechanisms save a return address on the stack before transferring control elsewhere.

2.5.1 Unconditional CALL

An unconditional CALL instruction always performs its transfer of control, regardless of any condition. It carries out three actions, in this order:

1. The address of the instruction immediately following the CALL — the return address — is pushed onto the stack, most significant byte first (or as the specific instruction set defines), and the Stack Pointer is decremented accordingly.
2. The Program Counter is loaded with the sixteen-bit address supplied as the operand of the CALL instruction — the entry point of the subroutine.

3. Execution continues from this new address exactly as though the processor had always been fetching instructions from there.

```
; Main program
MVI A, 07H
CALL 2500H ; save return address; jump to subroutine at 2500H
MOV B, A ; execution resumes here after the subroutine returns

; Subroutine at address 2500H
INR A ; increment the accumulator
RET ; pop return address back into the Program Counter
```

Step	Program Counter	Stack Pointer / Stack Contents	Action
Before CALL	points to CALL instruction	SP = 30F0H, stack empty	About to execute CALL 2500H
CALL executes	loaded with 2500H	SP = 30EEH; 30EEH–30EFH holds the return address	Return address pushed; jump taken
Subroutine runs	advances through 2500H, 2501H, …	unchanged	INR A executes
RET executes	reloaded from the stack	SP restored to 30F0H	Execution resumes after CALL

Figure 2.5 — Simulated execution of CALL and its matching RET, showing the round-trip of the Program Counter through the stack.

2.5.2 Conditional CALL

A conditional CALL instruction examines one specific bit of the status (flag) register and performs the same three-step save-and-jump sequence described above only if that bit is in the required state; otherwise, it does nothing at all beyond advancing the Program Counter past its own operand, exactly as an ordinary, non-branching instruction would. This allows a program to invoke a subroutine only when a particular arithmetic or logical condition has just been established by a preceding instruction.

Mnemonic	Condition Tested	Subroutine Is Called When…
CZ (Call if Zero)	Zero flag	the previous result was exactly zero (Z = 1)
CNZ (Call if Not Zero)	Zero flag	the previous result was non-zero (Z = 0)
CC (Call if Carry)	Carry flag	the previous operation produced a carry / borrow (CY = 1)
CNC (Call if No Carry)	Carry flag	the previous operation produced no carry / borrow (CY = 0)
CP (Call if Positive)	Sign flag	the previous result was positive (S = 0)
CM (Call if Minus)	Sign flag	the previous result was negative (S = 1)

Table 2.1 — Common conditional CALL mnemonics and the status-flag condition each one tests.

```
; Call an error-handling routine only if the last subtraction underflowed
SUB B       ; A ← A - B; Carry flag set if B > A
CC 3200H    ; call the error handler only if a borrow occurred
; --- execution continues here directly if CY = 0 ---
```

Practical example

A stock-control routine subtracts the quantity of an item being sold from the quantity currently held in inventory using SUB B. If the quantity sold exceeds the quantity in stock, this subtraction underflows and the Carry flag is set. Placing a CC (Call if Carry) instruction immediately afterwards, pointing at a small subroutine that displays an 'insufficient stock' message, means that the warning routine is invoked automatically and only when it is actually needed — the ordinary sale-processing path is not slowed down at all when stock is sufficient, since the conditional CALL then does nothing beyond a brief flag test.

Every conditional CALL has a natural timing consequence worth noting for later modules on pipelining: because the instruction may or may not transfer control, it may occupy a different number of machine cycles depending on the outcome of the flag test — typically fewer T-states when the call is not taken (since no stack-push or jump occurs) and more T-states when it is taken. This variability is one of the earliest and simplest examples of the kind of unpredictable instruction timing that later becomes a central concern in pipelined processor design.

Recap of the Lecture

This module has surveyed the instruction set as the working vocabulary of a processor, in four parts:

- An instruction set (ISA) is the fixed catalogue of opcodes and operand conventions a processor recognises; instructions are broadly grouped into data transfer, arithmetic, and program-control categories.
- Data transfer (copy) instructions such as MOV and MVI move values between registers, memory, and immediate data without altering the source or affecting the status flags.
- Arithmetic instructions — ADD, SUB, MUL and DIV — direct the ALU to compute new values and characteristically update the Zero, Carry, Sign and Parity flags, with multiplication and division often requiring double-width results spread across a register pair.
- The Interrupt Enable flag governs whether maskable interrupts are honoured, and is controlled directly by the EI and DI instructions, most commonly used to protect short critical sections of code from being interrupted midway through.
- CALL transfers execution to a subroutine while automatically saving a return address on the stack, matched by a later RET; conditional CALL variants (CZ, CNZ, CC, CNC, CP, CM, and others) perform the same save-and-jump sequence only when a specified status flag meets the required condition.

Laboratory Practicals and Exercises

As in Module 1, the following activities are best carried out using a microprocessor instruction-set simulator that allows single-step execution and inspection of registers, flags, memory, and the stack after every instruction.

Practical 1 — Simulating Data Transfer Instructions

- Load a register with an immediate value using MVI, then copy it into three other registers using successive MOV instructions, confirming after each step that the source register is unchanged.
- Load the H-L register pair with a valid memory address and use MOV A, M and MOV M, A to move a value from memory into the accumulator and back to a different address, verifying the final contents of both memory locations.

Practical 2 — Simulating Arithmetic Instructions and Their Flags

- Choose two 8-bit values whose sum exceeds 255 and execute ADD, recording the resulting Carry flag; repeat with two values whose sum is exactly zero (using two's-complement values) and record the Zero flag.
- Execute SUB with a subtrahend larger than the minuend and record the borrow (Carry) flag; explain, in your lab report, the two's-complement result that appears in the accumulator.
- If your simulator supports MUL and DIV, multiply two values whose product requires more than eight bits and confirm the result is correctly split across a register pair; then divide a value by zero (in simulation only) and record how your particular simulator represents the resulting error condition.

Practical 3 — Interrupt Enable / Disable

- Write a short program that raises a simulated hardware interrupt while interrupts are disabled (after DI) and confirm that it is not serviced until an EI instruction is later executed.
- Identify, from your simulator's or processor's documentation, whether EI takes effect immediately or only after the following instruction, and design a short test program that demonstrates the behaviour you found.

Practical 4 — CALL, Conditional CALL, and the Stack

- Write a subroutine that doubles the value in the accumulator, call it unconditionally from a main program, and trace the Stack Pointer and stack contents before the CALL, immediately after the CALL, and immediately after the matching RET.
- Modify the program so that the subroutine is invoked only via a conditional CALL (for example, CC or CZ) following an arithmetic instruction, and construct two separate test cases — one in which the condition is met and the subroutine executes, and one in which it is not.

End-of-Module Exercises

1. Distinguish between an opcode and an operand, and state how many of each a typical two-operand ADD instruction contains.
2. Explain why data transfer instructions such as MOV generally do not affect the processor's status flags, while arithmetic instructions such as ADD do.
3. Registers A and B hold the values 9CH and 45H respectively. Show the result and the state of the Carry flag after executing ADD B, and after executing SUB B, explaining any difference in how the Carry flag is interpreted in each case.

4. Explain why a native DIV instruction may not be available on every microprocessor, and describe, in outline, how division could be performed in software on a processor that lacks one.
5. Describe the sequence of internal actions that occur when a CALL instruction is executed, and explain how the matching RET instruction is able to resume execution at exactly the correct point.
6. Write, in the assembly notation used in this module, a short instruction sequence that calls a subroutine at address 4000H only if the result of a preceding SUB instruction was negative.
7. Explain, with reference to a critical section of your own choosing, why a programmer might deliberately execute DI before a sequence of instructions and EI immediately afterwards.

End of Module 2 — Instruction Set